МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования
«Московский авиационный институт (национальный исследовательский университет)»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ
№ 4

Дисциплина:

Операционные системы и архитектура компьютера

Тема:

Обмен данными между процессами через отображаемые в память файлы

Выполнил:

студент группы М8О-205БВ-25

Герасимов А.С.

Москва, 2026 г.

1. Цель работы

Освоить обмен данными между процессами через отображаемые в память файлы (mmap) и синхронизацию доступа к общей памяти с помощью семафоров. Вариант 2: родитель передаёт дочернему строки с числами float, дочерний считает сумму и пишет результат в файл.

2. Краткое описание архитектуры решения

Две отдельные программы — parent и child. Обе работают с одним файлом shared_memory_file, отображённым через mmap(MAP_SHARED), и видят одну структуру SharedData. parent.c: создаёт файл (open + ftruncate), отображает его (mmap), инициализирует два семафора и флаг is_done, запускает ребёнка через fork + execv. Читает строки с клавиатуры, кладёт в shared->buffer, будит ребёнка sem_post(sem_to_child), ждёт sem_wait(sem_to_parent). При Ctrl+D ставит is_done = 1, делает sem_post, ждёт waitpid, очищает ресурсы. child.c: получает fd и имя файла через argv, делает mmap того же файла, в цикле ждёт sem_wait(sem_to_child), проверяет is_done, парсит числа через strtod, пишет сумму в файл и консоль, делает sem_post(sem_to_parent).

Обработка ошибок: проверяются все syscall (open, ftruncate, mmap, sem_init, fork, execv, fopen, waitpid). Некорректный ввод (пустая строка, буквы, мусор) обрабатывается в child через флаг invalid.

2.1 Схема взаимодействия компонентов

Родитель читает строку с клавиатуры, копирует её в buffer разделяемой структуры SharedData и через sem_post(sem_to_child) будит ребёнка, после чего блокируется на sem_wait(sem_to_parent) и ждёт, пока буфер не будет прочитан. Ребёнок по sem_wait(sem_to_child) просыпается, проверяет is_done, парсит числа через strtod, пишет сумму в result.txt и консоль, после чего делает sem_post(sem_to_parent) и снова засыпает. Семафоры работают в режиме «пинг-понг»: всегда один равен 1, другой 0, поэтому родитель не перезапишет буфер, пока ребёнок его не прочитал. При Ctrl+D родитель ставит is_done = 1, делает sem_post(sem_to_child) и ждёт waitpid; ребёнок видит флаг и завершается, после чего родитель освобождает ресурсы (sem_destroy, munmap, close, unlink).

3. Листинг ключевых фрагментов кода

3.1 Структура разделяемой памяти (shared.h)

```
typedef struct {
    sem_t sem_to_child;
    sem_t sem_to_parent;
    int is_done;
```

```
    char buffer[MAX_STR_LEN];
} SharedData;
```

3.2 Инициализация mmap и семафоров (родитель)

```
      int fd = open("shared_memory_file", O_RDWR | O_CREAT | O_TRUNC,
0666);
      if (fd == -1) { perror("open"); return 1; }


      if     (ftruncate(fd,     sizeof(SharedData))     ==     -1)     {
perror("ftruncate"); close(fd); return 1; }


      SharedData *shared = mmap(NULL, sizeof(SharedData),
                                PROT_READ | PROT_WRITE, MAP_SHARED,
fd, 0);
      if (shared == MAP_FAILED) { perror("mmap"); close(fd); return 1;
}


      sem_init(&shared->sem_to_child,  1, 0);
      sem_init(&shared->sem_to_parent, 1, 0);
      shared->is_done = 0;
```

3.3 Передача fd ребёнку через execv

```
      pid_t pid = fork();
      if (pid == 0) {
          char fd_str[10];
          sprintf(fd_str, "%d", fd);
          char *args[] = {"./child", fd_str, fileName, NULL};
          execv("./child", args);
          perror("execv"); exit(1);
      }
```

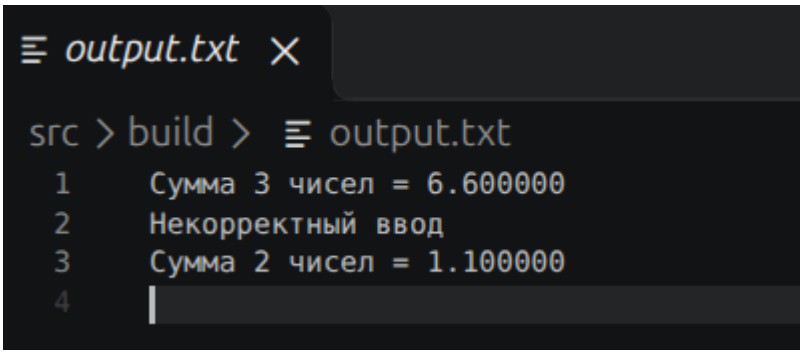
4. Результаты работы

4.1 Консольный вывод

```
art@honor:~/labs/sem3/os/lab4/src/build$ ./parent
Введите имя файла для записи результата: output.txt

Вводите строки (числа через пробел). Ctrl+D - выход
> 2.1 2.2 2.3
[Child] Сумма 3 чисел = 6.600000
> a b
[Child] Некорректный ввод
> 0.5 0.6
[Child] Сумма 2 чисел = 1.100000
>
```

4.2 Скриншоты результатов



5. Выводы

В ходе работы была разработана двухпроцессная программа на языке Си, в которой обмен данными между родительским и дочерним процессами организован через отображаемый в память файл (mmap с флагом MAP_SHARED), а синхронизация доступа к общему буферу — через два семафора. Реализована обработка некорректного ввода и системных ошибок, а также корректное освобождение ресурсов. Полученные результаты в файле result.txt и сообщения «Некорректный ввод» подтверждают работоспособность программы. В результате работы были освоены практические навыки использования memory-mapped files и межпроцессной синхронизации в Linux.